Sicherheitsprinzipien

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B
Version: 1.2.7

Kontrollfragen: https://delors.github.io/sec-klassische-sicherheitsprinzipien/kontrollfragen.de.md.html

Folien: [HTML] https://delors.github.io/sec-klassische-sicherheitsprinzipien/folien.de.md.html
[PDF] https://delors.github.io/sec-klassische-sicherheitsprinzipien/folien.de.md.html.pdf

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Klassische Sicherheitsprinzipien

Jerome Saltzer and Michael Schroeder, 1975

Überblick

Principle of Economy of Mechanism (aka Principle of Simplicity):

Die Sicherheitsmechanismen sollten so einfach wie möglich sein.

Es wird auch als Principle of Simplicity bezeichnet und fördert zum Beispiel die Korrektheit der Implementierung/Anwendung, da diese schneller verstanden wird und auch einfacher getestet werden kann. Weiterhin reduziert es die Angriffsfläche.

Beispiele:

- das Unix Datei-Rechte System
- Mailbox Kommunikation zwischen Systemen und Komponenten
- ...

Principle of Fail-Safe Defaults:

Standardmäßig sollte der Zugriff auf Ressourcen verweigert werden.

Principle of Complete Mediation:

Zugriffsanfragen eines Subjekts auf ein Objekt werden *jedes Mal* vollständig auf ihre Zulässigkeit hin überprüft.

◆ Bemerkung

Bei modernen Zero-Trust-Ansätzen gilt, dass jeder Zugriff erneut authentifiziert und autorisiert wird. Dies ist eine Umsetzung des *Principle of Complete Mediation*.

Bei der Entwicklung einer Serveranwendung besagt das *Principle of Complete Mediation* somit, dass bei jeder Anfrage zu überprüfen ist, ob der Nutzer die entsprechenden Rechte besitzt.

Principle of Least Authority (aka POLA)/ Principle of Least Privilege:

Jedes Programm und jeder Benutzer sollte nur die für seine Aufgabe unbedingt notwendigen Rechte besitzen.

Principle of Separation of Privilege:

Ein System sollte in mehrere POLA konforme Komponenten unterteilt sein. Sollte eine Komponente kompromittiert sein, dann sind die Möglichkeiten des Angreifers dennoch begrenzt. D. h. kritische Operationen sollten nur ausgeführt werden, wenn mehrere *unabhängige* Bedingungen erfüllt sind.
(Eng verwandt mit dem bzw. ergänzend zum POLA.)

Dieses Prinzip wird dann eingehalten, wenn ein Angreifer, der eine Komponente kompromittiert hat, nicht die Rechte erhält, die notwendig sind, um das gesamte System zu kompromittieren. Zum Beispiel ist es für Geldüberweisungen notwendig, diese auf zwei verschiedene Arten zu autorisieren.

▲ Achtung!

Privilege Separation (für Programme) sollte nicht mit dem hier beschriebenen Prinzip verwechselt werden. *Privilege Separation* liegt zum Beispiel dann vor, wenn ein Programm in zwei Teile aufgeteilt ist und ein Teil - zum Beispiel zum Zugriff auf Betriebssystemressourcen wie Sockets oder bestimmte Dateien - erhöhte Rechte benötigt als der Rest vom Programm. In diesem Fall erfolgt dann der Austausch zwischen den beiden

Teilen über eine wohldefinierte, minimale Schnittstelle, die die Rechte des ersten Teils auf das notwendige Minimum beschränkt.

Principle of Least Common Mechanism:

Die Sicherheitsmechanismen sollten über Nutzer (insbesondere aber auch Programme, die andere Programme nutzen) hinweg möglichst wenig gemeinsam haben.

(~  *Grundsatz des kleinsten gemeinsamen Mechanismus*)

Beispiel

Das Prinzip besagt zum Beispiel, dass die Mechanismen, die von mehreren Benutzern verwendet werden oder von dem mehrere Nutzer abhängen, minimiert werden sollten.

Das Prinzip kann/sollte auf ganz verschiedenen Ebenen angewendet werden:

- Z. B. sollten keine gemeinsamen Speicherbereiche verwendet werden in denen möglicherweise sicherheitsrelevantes Material vorgehalten wird. Es ist deswegen z. B. sinnvoll - wenn möglich - auf Implementierungen im Kernel zu verzichten und stattdessen auf User-Space-Implementierungen zu setzen.

TCP Connection Hijacking Angriffe werden bzw. wurden z. B. durch die Implementierung des TCP Stacks im Kernel ermöglicht (⇔ „Principle of Least Common Mechanism“).

- Z. B. sollten keine geteilten Passworte verwendet werden, um sich gegenüber einem System zu authentifizieren. (Dies bezieht sich sowohl auf die Passwörter einer Person als auch auf Passwörter über Personen und Systemgrenzen hinweg!)

Principle of Open Design:

Das Prinzip besagt, dass die Sicherheit eines Mechanismus nicht von der Geheimhaltung seines Designs oder seiner Implementierung abhängen sollte.
(Vgl. Kerckhoffs Prinzip: Die Sicherheit eines Kryptosystems sollte nur von der Geheimhaltung des Schlüssels abhängen.)

Principle of Psychological Acceptability:

Die Sicherheitsmechanismen sollten einfach zu verstehen und zu benutzen sein.

Principle of Isolation:

Die Sicherheitsmechanismen sollten so entworfen sein, dass Fehler in einem Teil des Systems nicht die Sicherheit des gesamten Systems gefährden; d. h. die einzelnen Komponenten sollten möglichst unabhängig voneinander sein und nur über wohldefinierte Schnittstellen miteinander kommunizieren und entsprechende Sicherheitsüberprüfungen durchführen.

Beispiel

1. Virtuelle Maschinen können genutzt werden, um Anwendungen in einer isolierten Umgebung auszuführen. Ein Angreifer, der eine Anwendung kompromittiert hat, kann somit nicht auf andere Anwendungen oder das Betriebssystem zugreifen.
2. Typischerweise kommuniziert zum Beispiel ein Basebandchip (WIFI, LTE, 5G, ...) mit dem Betriebssystem über eine minimale Schnittstelle über die nur Nachrichten übermittelt werden können, die leicht auf ihre Korrektheit überprüft werden können. Insbesondere erfolgt kein direkter Zugriff auf den Speicher des Betriebssystems.

Einen Angreifer ist es somit ggf. möglich den Basebandchip anzugreifen und ggf. zu kompromittieren, aber er kann nicht direkt auf das Betriebssystem zugreifen und Nachrichten,

die bereits auf Betriebssystem oder Anwendungsebene verschlüsselt werden, sind weiterhin sicher.

Principle of Modularity:

Die Sicherheitsmechanismen sollten so entworfen sein, dass sie unabhängig voneinander implementiert und geprüft werden können.

Principle of Layering:

Die Sicherheitsmechanismen sollten in Schichten organisiert sein.

Beispiel für ein Schutzsystem für Netzwerke, dass mehrere Schichten verwendet:

- einfache (und effiziente) Paketfilter auf unterster Ebene
- zustandsbehaftete Paketfilter auf der nächsten bzw. der Anwendungsebene

Principle of Least Astonishment:

Die Sicherheitsmechanismen sollten so entworfen sein, dass sie keine Überraschungen für die Benutzer bereithalten.

✎ Übung

1.1. Principle of Open Design

Benennen Sie ein historisches Verschlüsselungsverfahren, das gegen das *Principle of Open Design* verstoßen hat.

1.2. Verletzung

Stellen Sie sich vor, dass Sie als Pin (z. B. für ein Tablet) folgende Zahl verwenden wollen, diese aber abgelehnt wird (Leerzeichen dienen nur der Lesbarkeit):

```
3671 1197 4769
```

Während als Pin das folgende Passwort akzeptiert wird:

```
1364 7964 1364
```

Wie bewerten Sie dies?

※ Hinweis

Schauen Sie sich ggf. ein Pinpad an.

✎ Übung

1.3. Browser

Der Chrome-Browser (zum Beispiel) unterstützt die so genannten Isolierung von besuchten Webseiten. Bei dieser werden Seiten von verschiedenen Websites in unterschiedliche Prozesse aufgeteilt.

Welches Prinzip bzw. welche Prinzipien wird/werden hier umgesetzt?

1.4. Quantum Algorithmen für die Verschlüsselung

Zukünftige Verschlüsselungsalgorithmen, z. B. solche die auch im Zeitalter der Quantencomputer noch sicher sein sollen, werden häufig im Rahmen von offenen Wettbewerben entwickelt bzw. ausgesucht. Wie bewerten Sie dieses Vorgehen?

✎ Übung

1.5. Rechte von im Hintergrund laufenden Prozessen auf Servern

Es ist üblich, dass für Prozesse, die auf Servern im Hintergrund laufen, extra Nutzerkonten eingerichtet werden.

- Warum ist dies so?
- Welche Rechte sollten diese „Nutzer“ bekommen?
- Was sollte weiterhin beachtet werden?

Literaturverzeichnis

- R. E. Smith, "A Contemporary Look at Saltzer and Schroeder's 1975 Design Principles", in IEEE Security & Privacy, vol. 10, no. 6, pp. 20-25, Nov.-Dec. 2012, doi: 10.1109/MSP.2012.85
- J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," in Proceedings of the IEEE, vol. 63, no. 9, pp. 1278-1308, Sept. 1975, doi: 10.1109/PROC.1975.9939. (Online: Saltzer, Jerome H. and Schroeder, Michael D., "The Protection of Information in Computer Systems", April 1975, https://www.cs.virginia.edu/~evans/cs551/saltzer/)
- Saltzer, Jerome H., "Protection and control of information sharing in multics", in ACM SIGOPS Oper. Syst. Rev., vol. 7, no. 4, p. 119, Oct. 1973 doi: 10.1145/957195.808059
- Saltzer, Jerome H., "Protection and control of information sharing in multics", in Commun. ACM, vol. 17, no. 7, p. 119, July 1974 doi: 10.1145/361011.361067